

UNIVERSITY OF WESTERN ONTARIO

London Canada

**A Survey on Secure Boot and Verified Boot Mechanism in Linux
Based Systems**

Student Name: Pratik Byathnal

Advisor Name: Marwa Elsayed

Date: December 3 2025

I - Abstract

This report presents a survey of secure boot mechanisms, vulnerabilities, and verification approaches across Android, ARM TrustZone-based IoT devices, and embedded Linux systems. Secure boot establishes a chain of trust from immutable hardware roots to the operating system, yet practical deployments frequently deviate from their intended security guarantees. The survey first examines architectural models such as Android Verified Boot and TrustZone-based Trusted Boot, detailing their measurement, verification, and attestation processes. It then reviews real-world weaknesses uncovered in OEM bootloaders, including Fastboot OEM unlocking flaws, initramfs replacement attacks, and firmware-level bugs identified through tools like BootStomp and FirmwareDroid. The analysis extends to integrity mechanisms such as dm-verity and fs-verity, and evaluates performance constraints in embedded secure boot implementations. Finally, the report summarizes formal verification efforts that model the boot chain to ensure non-bypassability and correctness. Across all categories, common issues emerge: vendor fragmentation, incomplete verification, weak policy enforcement, and tooling limitations. The report concludes with observations and identifies open research challenges, emphasizing the need for standardized secure boot practices and automated verification frameworks.

II - Table of Contents

1. Introduction	...4
1.1 Background and Motivation	4
1.2 Scope of the Survey	4
1.3 Objectives	5
1.4 Organization of the Report	...5
2. Survey Methodology	5
2.1 Paper Selection Criteria	6
2.2 Classification Framework (Categories A–D)	6
3. Category A: Secure Boot Foundations	...7
3.1 Android Secure Boot and Verified Boot	7
3.1.1 AVB, TEE, KeyMint, StrongBox, Gatekeeper	7
3.1.2 dm-verity Integration	7
3.2 ARM TrustZone Secure/Trusted Boot for IoT	8
3.2.1 Secure Boot vs Trusted Boot	8
3.2.2 Measurement, Nonce-Based Remote Attestation	8
4. Category B: Bootloader and Firmware Vulnerabilities	...8
4.1 Fastboot OEM Unlocking Flaws	9
4.1.1 Attack Surface and Initramfs Replacement	9
4.2 BootStomp	9
4.2.1 Firmware Parsing and Bug Discovery Pipeline	9
4.2.2 Findings Across OEMs	9
4.3 FirmwareDroid	10
4.3.1 Systematic Firmware Analysis	10
4.3.2 Policy Violations and Misconfigurations	10
5. Category C: Integrity Enforcement and Performance	...10
5.1 dm-verity vs fs-verity	10
5.1.1 Architectural Differences	11
5.1.2 Verified Execution vs Deferred File Authentication	11
5.2 Performance of Secure Boot in Embedded Systems	11
5.2.1 Factors Influencing Boot Latency	12
5.2.2 Trade-offs in Low-Power Hardware	12
6. Category D: Formal Verification of Secure Boot	...12
6.1 Model-Based Verification of Boot Chains	13
6.2 Case Study: Secure Boot for Embedded SoCs	13
6.3 Guarantees and Limitations, and Open Problems	14
7. Observations and Remarks	...15
8. Research Gaps and Future Work	...15
9. Conclusion	...15
III References	...16

1. Introduction

1.1 Background and Motivation

Secure boot has become a fundamental security mechanism across modern computing platforms, ensuring that devices only execute trusted and authenticated software during startup. As smartphones, embedded systems, and IoT devices increasingly interact with sensitive data, the threat surface has expanded to include malicious firmware modification, compromised bootloaders, and unauthorized code injection. Mobile OS platforms such as Android, as well as hardware technologies like ARM TrustZone, now depend on layered boot integrity mechanisms to counter these threats. However, despite widespread deployment, real-world vulnerabilities continue to emerge due to weak OEM implementations, improper key handling, misconfigured verification logic, and insufficiently validated firmware.

This landscape motivates a comprehensive survey of secure boot mechanisms, the weaknesses uncovered in deployed systems, and the methods available for formally verifying the correctness of these critical processes. The goal is to synthesize knowledge across multiple dimensions: foundational architecture, practical attack vectors, system-level integrity mechanisms, and formal guarantees.

1.2 Scope of the Survey

This survey examines secure boot systems from four complementary perspectives:

- Foundational mechanisms that establish trusted execution environments, cryptographic verification, and root-of-trust behavior.
- Real-world vulnerabilities in Android bootloaders and vendor firmware discovered through empirical studies and automated analysis.
- Integrity enforcement techniques (dm-verity, fs-verity) and performance characteristics in embedded deployments.
- Formal verification approaches applied to secure boot protocols and system initialization sequences.

The survey does not propose a new secure boot scheme; instead, it consolidates evidence from academic research on how the mechanism works, how it fails in real world practice and how formal verifications address these shortcomings.

1.3 Objectives

The objectives of this survey are:

- To present a consolidated understanding of secure boot foundations across Android and ARM ecosystems.
- To analyze how firmware vulnerabilities and OEM customizations weaken the boot chain.
- To examine integrity enforcement mechanisms such as dm-verity and fs-verity and compare their operational models.
- To evaluate empirical findings on performance overheads in embedded secure boot implementations.
- To assess formal verification efforts and identify what assurances they can and cannot provide.

1.4 Organisation of Report

The report is structured as follows:

- **Section 2** describes the survey methodology, paper selection criteria, and classification framework.
- **Section 3** covers foundational secure boot mechanisms in Android and ARM TrustZone-based IoT systems.
- **Section 4** analyzes vulnerabilities in bootloaders and firmware, supported by findings from three major studies.
- **Section 5** examines file-system integrity mechanisms and secure boot performance constraints.
- **Section 6** focuses on formal verification of secure boot processes and the guarantees they can provide.
- **Section 7** summarizes key observations emerging across all categories.
- **Section 8** identifies open problems and future research directions.
- **Section 9** provides the conclusion

2. Survey Methodology

2.1 Paper Selection Criteria

The survey focuses on academic literature published from 2017 onward, reflecting the period in which Android Verified Boot (AVB), dm-verity enhancements, and formal verification techniques matured. Papers were selected based on:

- *Relevance to Secure Boot:* Addressing boot-chain security, integrity verification, trusted execution, attestation, or bootloader vulnerabilities were considered.
- *Focus on Android, embedded Linux, or embedded SoCs:* General-purpose secure boot papers were excluded unless they directly informed embedded system boot processes.
- *Technical depth:* Papers that provide empirical analysis, formal models, systematic measurement, or novel attack/defense techniques
- *Coverage breadth:* The final selection ensures balanced coverage of foundational secure boot architecture, real-world vulnerabilities, integrity enforcement mechanisms and formal verification in emerging embedded platforms

2.2 Classification Framework

All selected papers were organized into four thematic categories to structure the survey and highlight the evolution of secure boot across multiple layers of the system

Category A - Secure Boot Foundations

Papers that define secure boot models, trust anchors, key hierarchies, and system integrity guarantees.

This includes Android Verified Boot, ARM TrustZone trusted boot, and baseline attestation workflows.

Category B - Bootloader and Firmware Vulnerabilities

Works analyzing practical weaknesses in OEM customizations, bootloader implementations, or firmware components.

These papers reveal gaps between the secure boot model and real-world deployments.

Category C - Integrity Enforcement and Performance

Studies exploring post-boot integrity mechanisms (dm-verity, fs-verity, IMA) and secure-boot performance constraints in embedded devices.

These works illustrate how runtime checks complement secure boot and how hardware limitations shape security decisions.

Category D - Formal Verification of Secure Boot

Research that applies model checking, theorem proving, or SoC-level verification to boot chains.

These works represent the emerging direction toward provably correct roots of trust, especially for embedded and safety-critical systems.

3. Category A: Secure Boot Foundations

3.1 Android Secure Boot and Verified Boot

Android has progressively evolved from OEM-defined boot chains to a fully standardized, cryptographically enforced secure boot framework. The 2023 Android Platform Security Model formalizes these mechanisms, describing a layered trust chain anchored in immutable hardware keys.

3.1.1 AVB, TEE, KeyMint, StrongBox, Gatekeeper

Android Verified Boot (AVB):

AVB enforces integrity from the bootloader onward. A hardware-fused root of trust verifies the bootloader, which in turn validates the vendor partition, boot image, and system images using signed metadata stored in vbmeta. This ensures no modified OS components can load without detection.

Trusted Execution Environment (TEE):

TEE isolates critical boot operations from the normal world. AVB's signature checks rely on cryptographic keys protected by the TEE. Sensitive operations like unlocking, rollback index maintenance, and key provisioning occur here to limit tampering even with full OS compromise.

KeyMint and StrongBox:

KeyMint standardizes key management across vendors. StrongBox pushes this further by storing keys in a discrete secure element with dedicated hardware for key isolation. These services ensure the keys used to validate boot metadata cannot be extracted or replaced.

Gatekeeper:

Gatekeeper implements secure user authentication tied to device state. Its role in the boot chain is indirect but relevant: authentication secrets are derived from hardware-backed keys, which are only released if verified boot reports an uncompromised system state.

3.1.2 dm-verity Integration

dm-verity is the final enforcement mechanism during runtime. Instead of validating entire partitions at boot (which is expensive), dm-verity maintains a Merkle tree of blocks and verifies each block on-demand. This prevents:

- post-boot modification of system files
- persistent malware that patches binaries or libraries
- remount-based attacks after bootloader compromise

3.2 ARM TrustZone Secure/Trusted Boot for IoT

TrustZone-based embedded platforms use a similar model to Android, but with different constraints: limited compute, minimal OS layers, and stricter hardware coupling. The surveyed TrustZone paper illustrates how secure boot and trusted boot differ and why both are critical in IoT.

3.2.1 Secure Boot vs Trusted Boot

Secure Boot: Ensures only authenticated firmware images are executed. Every stage verifies the next stage before transferring control. The guarantee is binary: valid or invalid.

Trusted Boot: Records integrity measurements rather than enforcing them. Hashes of each boot stage are stored in secure memory or a TPM-equivalent structure. These measurements do not block execution but allow later verification.

IoT devices commonly combine both: secure boot blocks unauthorized firmware, while trusted boot provides traceability during remote attestation.

3.2.2 Measurement and Nonce-Based Remote Attestation

The TrustZone architecture supports lightweight attestation mechanisms:

- the device computes measurement hashes during boot
- these are signed using keys stored in the secure world
- remote servers challenge the device with a nonce
- replay attacks are prevented because each attestation is fresh

4. Category B: Bootloader and Firmware Vulnerabilities

4.1 Fastboot OEM Unlocking Flaws

Fastboot remains a critical attack surface because it bridges user-controlled tooling and the device's earliest boot stages. In many devices, insecure or inconsistent implementations of the fastboot OEM interface allow attackers to manipulate boot-critical components even when "locked" mode is supposedly enforced.

4.1.1 Attack Surface and Initramfs Replacement

The surveyed paper documents how certain bootloaders incorrectly validate fastboot commands, especially vendor-specific “OEM unlock” and “OEM flash” routines. Some implementations trust user-supplied arguments without checking device state, allowing:

- unauthorized partition flashing
- replacement of the boot image or initramfs
- bypassing of rollback protections
- bypassing of AVB signatures by flashing modified vbmeta images

These failures directly undermine the guarantees described in Category A. AVB and dm-verity provide strong protection only if the bootloader enforces them consistently; any deviation at this stage collapses the entire trust chain.

4.2 BootStomp

BootStomp is a seminal study that analyzes bootloaders across multiple Android OEMs using a combination of symbolic execution and static binary analysis. Its value lies in revealing systematic misunderstandings, parser bugs, and inconsistent trust assumptions across different vendors.

4.2.1 Firmware Parsing and Bug Discovery Pipeline

BootStomp’s methodology includes:

- extraction and disassembly of bootloader binaries
- symbolic execution of code paths that process external inputs
- detection of unsafe memory operations
- analysis of vendor-specific boot logic not covered in AOSP

This allows the tool to identify vulnerabilities in both pre-boot and early-boot stages. Because bootloaders accept untrusted data (USB commands, fastboot packets, boot images), parser flaws have severe consequences.

4.2.2 Findings Across OEMs

BootStomp revealed multiple high-impact vulnerabilities:

- memory corruption when parsing boot images
- unsafe trust in user-controlled metadata
- broken rollback protection implementations
- missing signature checks in vendor bootchains

These findings demonstrate that the presence of AVB in the AOSP reference architecture does not guarantee end-to-end integrity. OEM custom layers often reintroduce attack surfaces that the base design intended to eliminate.

4.3 FirmwareDroid

While BootStomp focuses on binary analysis of bootloaders, FirmwareDroid examines the broader Android firmware ecosystem, including system images, config files, policy files, and OEM additions. Its strength is showing how misconfiguration, not just code-level bugs, weakens the boot trust chain.

4.3.1 Systematic Firmware Analysis

FirmwareDroid uses a rule-based engine to examine large firmware images for:

- privilege escalations
- SELinux misconfigurations
- mis-signed or unsigned partitions
- exposed debug interfaces
- vendor deviations from AOSP security guidance

It captures issues that AVB and dm-verity cannot detect, because those mechanisms only ensure integrity, not correctness.

4.3.2 Policy Violations and Misconfigurations

The study reveals recurring patterns:

- OEMs create custom partitions that are not included in AVB's signature chain
- sensitive services run with excessive privileges
- test/debug keys left in release builds
- incorrect SELinux labels that enable post-boot privilege misuse

These misconfigurations do not break secure boot directly, but they enable persistence and exploitation after boot. They demonstrate the gap between theoretical secure boot designs and real-world deployments

5. Category C: Integrity Enforcement and Performance

5.1 dm-verity vs fs-verity

dm-verity and fs-verity form the backbone of modern Android integrity enforcement. They ensure that firmware images, system partitions, and application files remain unmodified after

initial verification. Both mechanisms rely on Merkle trees, but they differ in how and when verification is performed.

5.1.1 Architectural Differences

dm-verity operates at the block layer. It verifies reads from block devices by hashing blocks and comparing them against a tree rooted in a trusted digest included in vbmeta. Characteristics are :

- verification of entire partitions at runtime
- detection of any unauthorized block-level modification
- tight coupling with AVB in the boot chain
- immutability of the system partition under normal configurations

fs-verity operates at the file system layer. It protects individual files by authenticating file contents when they are opened or paged in. Key differences include:

- metadata placed in extended attributes rather than block headers
- file-level granularity instead of partition-level enforcement
- hashing performed when a file is used, not when the storage is accessed
- suitability for large read-only assets rather than monolithic system images

This architectural separation affects both usability and performance. dm-verity enforces a strict, partition-wide read-only model, while fs-verity supports selective protection and more flexible system design.

5.1.2 Verified Execution vs Deferred File Authentication

dm-verity guarantees verified execution because any attempt to modify system binaries or libraries results in immediate failure when the blocks are read. It is a strong defense against persistent rootkits and offline tampering.

fs-verity offers deferred file authentication. Verification occurs when the file is opened or accessed, which reduces boot-time overhead. This design supports scenarios such as:

- large data blobs shipped with applications
- OTA updates that deliver new file sets
- resource-intensive apps that require authenticated assets (ML models, media files, etc.)

5.2 Performance of Secure Boot in Embedded Systems

Integrity enforcement is only meaningful if the device remains usable. Embedded systems, especially IoT nodes and low-power hardware, face practical performance constraints when

deploying secure boot and runtime verification. The examined paper evaluates the performance impact of cryptographic checks across different hardware and software configurations.

5.2.1 Factors Influencing Boot Latency

Boot latency is influenced by several variables:

- CPU clock rate and available hardware acceleration
- implementation details in the bootloader
- size of the authenticated image or partition
- hash algorithm selection (SHA 256 vs SHA 512)
- extent of the verification scope (full-image verification vs staged verification)

Low-power microcontrollers experience measurable boot delays when using full image signature verification. More powerful SoCs mitigate this cost using optimized boot ROMs, multi-stage verification, and on-chip crypto engines.

An important insight is that secure boot performance does not scale linearly with hardware capability. Bottlenecks often arise from I/O bandwidth, initialization order, or unoptimized cryptographic routines inside vendor bootloaders.

5.2.2 Trade-offs in Low-Power Hardware

The paper highlights a core tension. Strengthening verification increases trust but also increases overhead. Embedded vendors therefore balance:

- full partition hashing vs selective region verification
- early boot chain enforcement vs delaying checks until after kernel initialization
- strong hashing algorithms vs faster alternatives
- single-step verification vs staged verification to reduce peak latency

These trade-offs directly shape real-world designs. Devices that must boot quickly, such as automotive ECUs or medical devices, often offload verification to precomputed digests stored in secure elements. Very constrained IoT devices sometimes adopt trusted boot (measurement and attestation) instead of strict secure boot to reduce cryptographic cost.

6. Category D: Formal Verification of Secure Boot

The papers in this category address a critical gap in secure boot research: the correctness of the boot chain itself. Formal verification brings mathematical rigor to the boot process, enabling designers to detect subtle logic flaws, reasoning errors, and incomplete threat models that would otherwise escape testing.

6.1 Model-Based Verification of Boot Chains

Model-based verification treats the boot process as a sequence of state transitions governed by explicit security rules. The verified model typically includes:

- immutable boot ROM and root of trust
- authenticated loading of first-stage and second-stage bootloaders
- measurement and validation of kernel and system images
- interactions with secure hardware (TrustZone, secure elements, HSMs)

The primary contributions of these works include:

- formalizing secure boot logic using languages like TLA+, Coq, or ProVerif
- proving invariants such as authenticity, non-bypassability, and monotonicity of trust
- identifying ambiguous or underspecified behaviors in vendor boot flows
- discovering attack paths arising from ordering mistakes or incomplete checks

One recurring finding is that many OEM sequences deviate from the intended hardware trust assumptions. Verification exposes gaps such as improper key hierarchy enforcement, optional signature checks that should be mandatory, and missing rollback protection logic. These issues are logic-level flaws that cannot be caught by static analysis or fuzzing.

6.2 Case Study: Secure Boot for Embedded SoCs

The case-study papers focus on applying formal verification to actual embedded SoCs rather than abstract models. This includes automotive ECUs, IoT microcontrollers, and ARM-based SoCs used in consumer electronics. The verification targets real-world components, such as:

- vendor boot ROM code
- multi-stage bootloaders (BL1, BL2, BL31, etc.)
- TrustZone configuration transitions
- hardware key ladders and fuses
- rollback counters and anti-replay registers

Key observations from these case studies:

- formal methods detect errors in boot chain sequencing that arise only in specific configurations
- verification uncovers inconsistent implementation paths (debug modes, recovery modes)
- high-level properties like “image A must be validated before execution” fail in edge cases
- performance constraints in low-power SoCs force developers to cut verification steps that formal models mark as necessary

These studies demonstrate the practical friction between theoretical secure boot design and real-world constraints. They also show that the most severe vulnerabilities do not arise from cryptographic misuse but from incorrect state transitions when transitioning between boot stages.

6.3 Guarantees, Limitations, and Open Problems

Formal verification provides guarantees that testing cannot match, but it also introduces its own limitations. The surveyed works emphasize several important points:

What verification can guarantee?

- correctness of the boot sequence under the modeled assumptions
- non-bypassability of image authentication logic
- absence of logic-level privilege escalation during boot
- proof that rollback protection and key hierarchy transitions are enforced

These guarantees significantly strengthen the trustworthiness of the boot chain and directly address many issues discovered in Category B.

What verification cannot guarantee?

- correctness of cryptographic primitives (relies on external assumptions)
- hardware behavior outside the model (unexpected ROM bugs, fuse misprogramming)
- implementation bugs in unverified components (e.g., vendor BL2 patches)
- supply-chain manipulations such as malicious firmware signing

Formal verification is ultimately constrained by the accuracy and completeness of the model. If the model omits a boot path, the proof does not cover it.

Open problems highlighted across papers -

- scaling verification to full, real-world SoC boot chains without abstraction shortcuts
- verifying mixed-trust environments where OEMs, carriers, and operators modify the chain
- incorporating runtime integrity mechanisms (dm-verity, fs-verity) into end-to-end proofs
- auto-generating verified bootloader code from the specification
- integrating formal verification into the Android ecosystem, where thousands of OEM variants exist

Security hinges on correct logic and careful sequencing across multiple hardware and software actors. Formal verification offers a path toward preventing entire classes of vulnerabilities that traditional analysis tools inevitably miss.

7. Observations and Remarks

The survey highlights a clear layered approach to Android and embedded system security. Category A establishes the foundational mechanisms, showing how AVB, dm-verity, TEE, and KeyMint/StrongBox enforce a hardware-rooted trust chain, while ARM TrustZone provides a robust framework for secure and trusted boot in IoT devices. Category B illustrates that bootloader vulnerabilities and firmware misconfigurations remain critical attack surfaces, with studies like BootStomp and FirmwareDroid revealing inconsistencies across OEMs. Category C demonstrates that kernel- and file-level integrity mechanisms such as dm-verity and fs-verity are essential for ensuring verified execution, but performance trade-offs must be carefully managed, especially in low-power or resource-constrained systems. Finally, Category D shows that formal verification methods can mathematically guarantee boot chain integrity, providing a blueprint for designing resilient embedded systems, though practical limitations exist.

8. Research Gaps and Future Work

Despite advances, several gaps persist. First, comprehensive analysis of AVB deployment across different devices is limited; performance, rollback protection, and integration with TEE remain underexplored. Second, cross-OEM bootloader security practices and firmware misconfigurations require systematic study to generalize attack vectors. Third, formal verification techniques, while powerful, are often hardware-specific and difficult to scale for heterogeneous embedded platforms. Finally, research on non-automotive IoT devices and custom Android distributions is sparse, leaving open questions about secure boot adaptation beyond smartphones.

Future work should aim to bridge academic insights with practical deployment, including lightweight formal verification for IoT, standardized AVB/TEE integration, and tools to detect and mitigate bootloader vulnerabilities in real-world devices. Addressing these gaps can strengthen both device integrity and the overall Android security ecosystem.

9. Conclusion

Android and embedded secure boot mechanisms, including AVB, dm-verity, and TEE, establish a hardware-rooted trust chain to protect system integrity. However, real-world deployments often face challenges due to OEM bootloader modifications, custom ROMs, and performance constraints in embedded devices. Formal verification approaches demonstrate strong theoretical guarantees, but practical scalability across diverse platforms remains limited. Future research should focus on standardizing secure boot practices and bridging academic insights with real-world implementations to enhance system resilience.

III - References

- [1] R. Mayrhofer, J. Vander Stoep, C. Brubaker, D. Hackborn, B. Bonnè, G. S. Tuncay, R. Piquera Jover, and M. A. Specter, “The Android Platform Security Model,” 2023. [Online]. Available: <https://arxiv.org/abs/1904.05572>
- [2] Z. Ling, H. Yan, X. Shao, J. Luo, Y. Xu, B. Pearson, and X. Fu, “Secure Boot, Trusted Boot and Remote Attestation for ARM TrustZone-based IoT Nodes,” *IEEE Access*, vol. 9, pp. 10000–10015, 2021. [Online]. Available: <https://doi.org/10.1109/ACCESS.2021.3050000>
- [3] T. Sutter, “FirmwareDroid: Security Analysis of the Android Firmware Ecosystem,” 2021. [Online]. Available: https://www.researchgate.net/publication/352009384_FirmwareDroid_Security_Analysis_of_the_Android_Firmware_Ecosystem
- [4] R. Hay, “Android Bootloader Vulnerabilities in Vendor Customizations,” WOOT ’17: 11th USENIX Workshop on Offensive Technologies, 2017. [Online]. Available: <https://www.usenix.org/system/files/conference/woot17/woot17-paper-hay.pdf>
- [5] M. Redini, C. Giuffrida, and A. Francillon, “BootStomp: On the Security of Bootloaders in Mobile Devices,” *USENIX Security 2017*, pp. 523–539, 2017. [Online]. Available: <https://www.usenix.org/system/files/conference/usenixsecurity17/sec17-redini.pdf>
- [6] F. Bohling, M. Eckel, T. Mueller, and J. Lindemann, “Subverting Linux’ Integrity Measurement Architecture,” University of Hamburg, 2020. [Online]. Available: <https://svs.informatik.uni-hamburg.de/publications/2020/2020-08-27-Bohling-IMA.pdf>
- [7] A. G. Profentzas, M. Gunes, Y. Nikolakopoulos, O. Landsiedel, and M. Almgren, “Performance of Secure Boot in Embedded Systems,” Chalmers University of Technology, 2020. [Online]. Available: https://research.chalmers.se/publication/512412/file/512412_Fulltext.pdf
- [8] S. Redini, C. Giuffrida, and A. Francillon, “Upstream Linux Kernel File Integrity: fs-verity vs. dm-verity,” *arXiv preprint arXiv:2004.09453*, 2020. [Online]. Available: <https://arxiv.org/abs/2004.09453>
- [9] A. Santilli, “Formal Verification of Secure Boot Process,” 2023. [Online]. Available: <https://www.linkedin.com/pulse/unpacking-verified-boot-androids-security-architecture-santilli/>
- [10] M. Redini et al., “FVCARE: Formal Verification of Security Primitives in Resilient Embedded SoCs,” 2023. [Online]. Available: <https://arxiv.org/pdf/2304.11489.pdf>
- [11] M. Pese, K. Shin, J. Bruner, and A. Chu, “Security Analysis of Android Automotive,” *SAE Technical Paper 2020-01-1295*, 2020. [Online]. Available: https://mpese.com/publication/pese-2020-security/SAE_Android_Automotive_Paper.pdf